

GitHub Issue Duplicate Detection

Final Project Report

Team Members:

- Seth Strzechowski (sstrz2)
- Sezim Zamirbekova (szami2)
- Priyansh Patel (ppp9)

Overview

This tool detects duplicate issues in GitHub repositories using semantic similarity. Instead of simple keyword matching, it understands the meaning behind issue descriptions to find related bug reports and feature requests. This saves maintainers of large open-source projects from manually sifting through thousands of issues to find duplicates.

The system works in two phases. First, it indexes existing issues from a repository by converting their text into vector embeddings. Second, when given a new issue, it searches the index to find the most similar existing issues and ranks them by similarity score.

How to use the software

Setup:

1. Clone the repository and create a Python virtual environment:

```
python3 -m venv venv  
source venv/bin/activate
```

2. Install dependencies:

```
pip install -r requirements.txt
```

3. Set your GitHub token as an environment variable. This is required because the GitHub API has strict rate limits without authentication:

```
export GITHUB_TOKEN="your_token_here"
```

Building the index:

Run the build script to fetch issues from a repository and create the vector index:

```
python scripts/build_index.py --repo Homebrew/brew
```

The script fetches issues from the repo (up to 60 pages, 100 issues per page), cleans the text, generates embeddings using sentence-transformers, and saves everything to the data folder. It creates four files: `issues_raw.json` (the raw API response), `embeddings.npy` (the vector embeddings), `meta.json` (issue metadata like titles and URLs), and `ids.json` (issue numbers).

Note: We recommend using Homebrew/brew for testing as it has a manageable number of issues. Larger repositories like `microsoft/vscode` or `microsoft/TypeScript` can exceed the GitHub API rate limit (5000 requests/hour) during evaluation, since the evaluation scripts need to fetch comments for each issue to find known duplicates.

Querying for duplicates:

To find duplicates for a new issue, use the CLI tool:

```
python scripts/query_cli.py
```

Enter the issue title and body when prompted. The tool returns the top 5 most similar issues with their similarity scores and URLs.

Running hierarchical clustering:

For better duplicate grouping, run the clustering script:

```
python scripts/cluster_issues.py --method ward --max-clusters 100
```

This groups similar issues into clusters using hierarchical clustering. The output includes a JSON file with cluster assignments and a markdown summary showing all clusters and their members.

Evaluation:

To measure how well the system detects known duplicates:

```
python scripts/evaluate.py --repo Homebrew/brew
```

This script fetches issues, parses comments to find "duplicate of #NNNN" references, and calculates Recall@k and MRR@k metrics. These metrics tell us how often the true duplicate appears in the top k results.

To evaluate the clustering approach specifically:

```
python scripts/evaluate_clusters.py --repo Homebrew/brew
```

How the software is implemented

The codebase is organized into a main package (`src/dedupe_detector`) containing the core modules, and a `scripts` folder with command-line tools that tie everything together.

GitHub Client (`github_client.py`):

Handles all interaction with the GitHub API. It fetches issues in batches of 100 per page to minimize API calls. The client also parses issue comments to find duplicate references by looking for patterns like "duplicate of #1234" or "dup of #5678". Rate limiting is handled automatically by checking the `X-RateLimit-Reset` header and sleeping until the limit resets.

Text Preprocessing (`preprocess.py`):

Cleans issue text before embedding. It removes code blocks (which would add noise), strips URLs, and combines the title and body into a single string. The text is truncated to 2000 characters to keep embedding times reasonable while preserving the most important information.

Embedding (`embed.py`):

Uses the `sentence-transformers` library with the `all-MiniLM-L6-v2` model. This model is small and fast while still producing good quality embeddings. The embeddings are normalized so that cosine similarity can be computed as a simple dot product.

Vector Index (`index.py`):

Implements exact cosine similarity search using NumPy. Given a query vector, it computes the dot product against all stored vectors and returns the top-k most similar. This approach is simple and works well for datasets up to tens of thousands of issues.

Detection (`detection.py`):

The `Detector` class combines the embedder and index to provide a query interface. Given an issue title and body, it cleans the text, generates an embedding, searches the index, and returns ranked results with metadata. It supports excluding a specific issue from results, which is useful during evaluation when you don't want an issue to match itself.

Clustering (`clustering.py`):

Implements hierarchical agglomerative clustering using `scipy`. The `cluster` method takes the embedding matrix and groups issues using linkage algorithms like Ward, complete, or average.

Ward minimizes variance within clusters and tends to produce balanced groups. The clustering results map each issue to a cluster ID, making it easy to find all related issues.

Cluster Detector (`cluster_detector.py`):

Combines the vector search and clustering approaches. When querying, it first finds the nearest issue in the index, then retrieves all issues in that same cluster. This narrows down the search space to a more relevant subset before ranking by similarity.

Evaluation (`eval.py`):

Calculates standard information retrieval metrics. Recall@k measures whether any true duplicate appears in the top k results. MRR (Mean Reciprocal Rank) measures how high the first true duplicate ranks. These metrics are averaged across all test queries to get overall performance numbers.

Technical details

The embedding model (all-MiniLM-L6-v2) produces 384-dimensional vectors. These capture semantic meaning, so issues about the same topic will have similar vectors even if they use different words.

Cosine similarity ranges from -1 to 1, where 1 means identical vectors. In practice, duplicate issues typically have similarity scores above 0.7, while unrelated issues score below 0.3.

The hierarchical clustering uses Euclidean distance on the normalized embeddings. Ward linkage tends to work best because it creates compact, similarly-sized clusters. The `max_clusters` parameter controls granularity, with more clusters meaning smaller and more specific groups.

The GitHub API allows 5000 requests per hour with authentication. Fetching issues is cheap (about 60 API calls for 6000 issues at 100 per page). However, evaluation is expensive because it fetches comments for each issue to find "duplicate of" references. The system handles rate limiting gracefully by detecting 403 responses and waiting for the limit to reset.

Results and observations

Testing on Homebrew/brew, the basic vector search achieves reasonable recall on known duplicates. The clustering approach helps by grouping related issues together, making it easier to browse potential duplicates rather than just getting a ranked list.

The main limitation is that duplicate detection depends heavily on text similarity. Issues that describe the same bug in very different ways may not be detected. Additionally, some duplicates are only identifiable by looking at screenshots or reproduction steps, which our text-based approach cannot capture.

Future improvements

Potential enhancements include incorporating issue labels and other metadata into the similarity calculation, using larger embedding models for better accuracy, and building a web interface for easier interaction. The clustering could also be made dynamic, updating incrementally as new issues are created rather than requiring a full rebuild.